

# Unveiling Active Learning Strategies: Similarities, Applicability to Unlabeled Data and Influence of Initial Performance

Elia Rühle  
TU Dresden

Vincent Melisch  
TU Dresden

Tony Ray Marx  
TU Dresden

Wilhelm Pertsch  
TU Dresden

Anna Lopatkina  
TU Dresden

Paul Böse  
TU Dresden

Julius Gonsior  
TU Dresden

Anja Reusch  
TU Dresden

Wolfgang Lehner  
TU Dresden

## Abstract

Active Learning (AL) has emerged as a prominent approach to optimize the labeling process in Machine Learning (ML) tasks. Over the time, numerous frameworks and implementations of AL strategies have been developed to address the task of effective labeling. In this paper, we present a comparative analysis of various AL strategies. We discuss the similarities among popular AL strategies and propose a framework for strategy recommendation. Additionally, we analyzed the impact of the initial AL-strategy performance on the overall performance. Our findings contribute to understanding the effectiveness of AL and provide valuable guidance for selecting the right AL approach for different conditions.

## 1 Introduction

Active Learning (AL) has improved the field of Machine Learning (ML) by enabling efficient data labeling and improving model performance. AL describes an iterative process which begins with a sparsely labeled or unlabeled data set to train a ML model. A query strategy selects a batch of informative instances of this data set and sends them to an expert for labeling. By retrieving the new labels, the ML model can be updated. This process continues until a predefined stopping criterion is reached. The advantage of AL is that only data which yields the most promising increase in performance is labeled. However, the abundance of such AL strategies and frameworks has created a need for comparative evaluations to guide researchers and practitioners in selecting the most suitable approach.

### 1.1 Outline

In this work, we conduct a comprehensive analysis of AL strategies with a strong focus on three major topics. First, we investigate the similarities between different AL strategies as described in

**Section 2.** To enable a valuable guiding for AL strategy selection, **Section 3** introduces a decision-tree-based and vector-space-based framework for strategy recommendation. Finally, we analyze the relationship between the starting performance and overall performance of AL strategies in **Section 4**. Finally we present a conclusion in **Section 5** and future work in **Section 6**.

### 1.2 Related Work

Our work is based on five widely used AL frameworks: LIBACT by Yang et al. (2017), ALiPy by Tang et al. (2019), Small-text by Schröder et al. (2023), scikit-activeml by Kottke et al. (2021) and Active Learning Playground<sup>1</sup> by Google. Additionally, Gonsior et al. (2022) introduced greedy based AL strategies that are considered for benchmarking. The performance of several functions of these frameworks were evaluated on different data sets with different metrics. The utilization of meta-features for model recommendation selection was first introduced by Brazdil et al. (2022) in their seminal work on meta-learning. Their approach had a profound impact on the development of AL-strategy recommendation frameworks. They extensively investigated various categories of meta-features, encompassing simple, statistical, information-theoretic and land marking measures. These meta-features were leveraged to accurately characterize data sets and facilitate the process of model selection. Notably, our study drew inspiration from suggestions of Brazdil et al. (2022) regarding the use of meta-signatures, which were employed in the construction of our own strategy recommendation framework.

### 1.3 Preliminaries

The underlying data for our analysis consist of multiple metrics taken from 52 different AL strategies

<sup>1</sup><https://github.com/google/active-learning>

applied to 39 data sets. A single experiment is defined by a strategy applied for a fixed number of queries on a data set with a specific start point and number of labeled points in each iteration. Furthermore, the applied ML model as well as the train and test bucket size is specified. The metrics contain single values and sometimes lists of values, which can be a collection of single metric values or indices of labeled data points. Most of the metrics data are normalized in the interval  $[0, 1]$ . The successful experiments and their corresponding hyperparameters can be found in the project’s code<sup>2</sup>.

## 2 Similarities of AL-strategies

The AL frameworks contain strategies, each with the goal to optimize the performance of a certain ML model. Since different approaches are used within the frameworks and performance is not uniquely defined, we look at whether there are AL strategies across all frameworks that often deliver similar results for a selection of metrics and therefore can be considered similar. We show that there are AL strategies which are very similar to each other. Especially, similar approaches from different frameworks show a high agreement. We also show that there are some AL strategies which could only rarely achieve similar results and therefore can rather not be considered similar.

### 2.1 Data Preselection

In order to obtain a high rate of comparability of the data, we calculate the number of unique experiments sampled for each AL strategy. Because of missing experiment data for some AL strategies, we only use 37 AL strategies that have at least 25.000 sampled experiments for comparison. Further, we calculate the sets of all hyperparameter tuples which identify single experiments for a single strategy. The intersection over all sets results in 4034 unique combinations of hyperparameters for which an experiment was sampled for every strategy. Since not every metric contains numerical data which can be used to detect similarities, we select 93 suitable metrics<sup>3</sup>.

### 2.2 Data Interpolation

A single experiment is ideally sampled with 100 AL cycles and one numerical metric entry for every

cycle. If an experiment terminates regularly or irregularly before the 100 cycles are reached, the remaining cycles are filled with a NaN value. To avoid NaN entries for the experiments, we replaced them by the last valid value of the respective experiment. Some metrics contain lists of values for one AL cycle. In this case, we replace the lists by their average value. As the final result all metrics contain a single numerical value for every AL cycle.

### 2.3 Clustering

To detect similarities between the AL strategies, we clustered the data. For the clustering we used the scikit-learn implementation of the k-Means algorithm with 10 initial centers, the k-means++ cluster centroid initialization and a convergence tolerance of  $10^{-4}$ . Further, we restricted the algorithm to a maximum of 50 iterations for a single run. For every selected metric and data set from 2.1 we iterate over the hyperparameters of experiments which were sampled on the data set and extract a 100 dimensional vector, representing the 100 AL cycles for each AL strategy. This leads to a collection of 37 vectors for every clustering. To track the results of the clustering we define a matrix  $M = 0_{37 \times 37}$ . Every time two AL strategies  $s_1, s_2 \in \{1, \dots, 37\}$  have the same label we set  $M(s_1, s_2) = M(s_1, s_2) + 1$ . This approach allows us to keep track of how often different AL strategies are in the same cluster. To investigate the impact of a reduction of dimensions, we run the clustering both with and without Principal Component Analysis (PCA). For PCA we use the scikit-learn implementation with 4 components and a full Singular Value Decomposition (SVD) solver.

### 2.4 Results

It is determined that the application of PCA has no noticeable influence on the clustering results. As shown in **Figure 1**, one can observe some expected similarities of 98–99% between AL strategies with similar implementation. Further, there are strong similarities of 75–79% between the AL strategies (27), (28) and (29), 75–78% for (9), (10) and (11) as well as 74–77% for (30), (34) and (36). Additionally the AL strategies (14), (16), (18) and (19) are similar in 36–49% of the cases. It is interesting to note that these AL strategies, which are remarkably similar to each other, always originate from the same framework, with the exception of those with a similarity of over 98%. Across differ-

<sup>2</sup>[https://github.com/paul1995tu/Data-Mining/blob/main/remake/05\\_done\\_workload.csv](https://github.com/paul1995tu/Data-Mining/blob/main/remake/05_done_workload.csv)

<sup>3</sup><https://github.com/paul1995tu/Data-Mining/blob/main/remake/config/data.yaml>

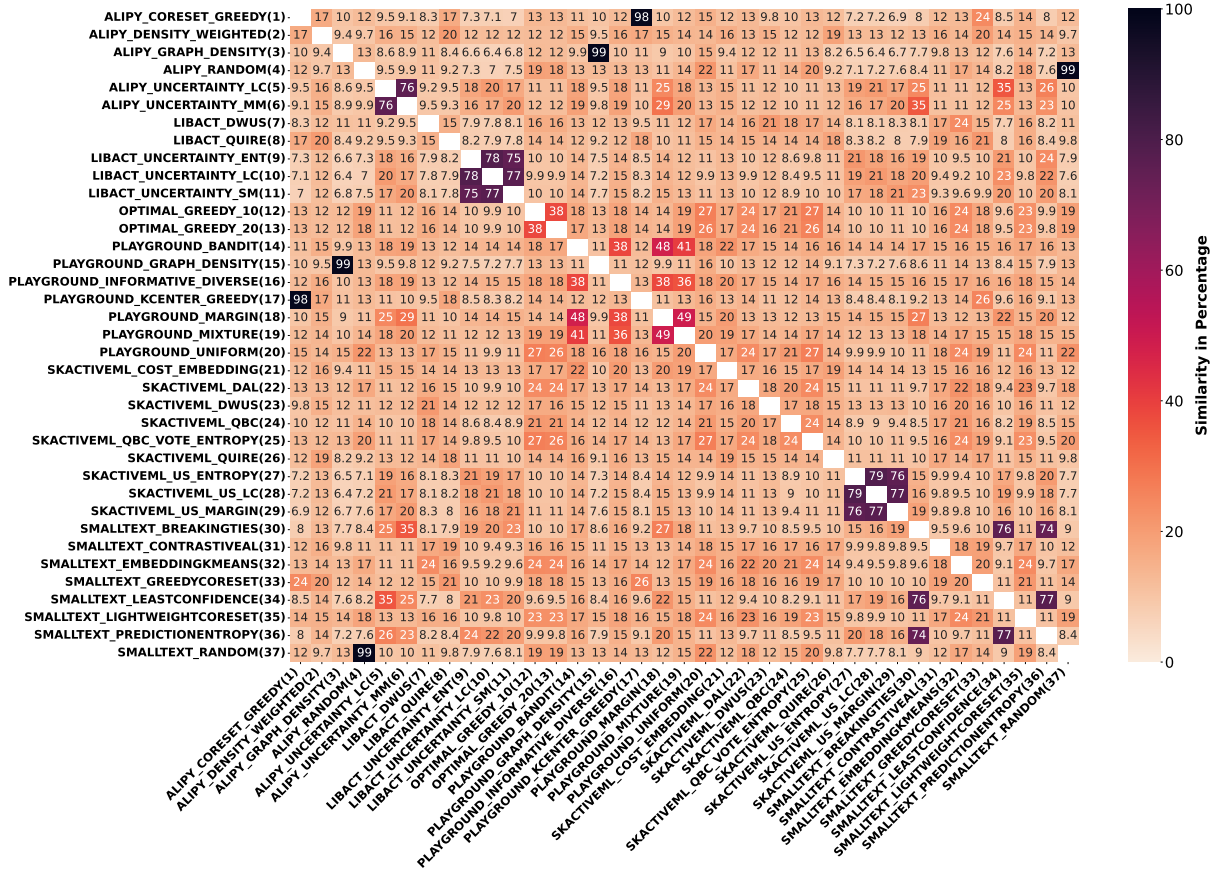


Figure 1: Normalized similarity matrix for all evaluated AL strategies.

ent frameworks the AL strategies (12), (13), (20), (22), (25), (32) and (35) achieve the best similarity with at lowest 23% and at highest 38% closely followed by (5), (6), (18), (30), (34) and (36) with 25 – 35%. At least in 24 – 27% the AL strategies (20), (25) and (32) are similar. A similar value is also achieved by (1) and (33) with 25%: Slightly below these values we also noted the similarity of (7) and (32) with 21%. There are a lot of AL strategies which have a similarity under 20%. These are rather to be classified as not similar. For a detailed look at the similarity scores consider **Figure 1**.

### 3 Applicability to Unlabeled Data Sets

The objective of this section is to develop predictors that are capable of suggesting an optimal strategy for a data set, based on data set characteristics and an input vector that encompasses (1) the *data set* in focus, (2) the *batch size* and (3) a chosen *metric*. During the training phase of the predictors, a score is calculated for each potential input vector for every AL strategy. This results in a ranking which identifies the best-suited AL strategies for a particular data set, batch size and metric. In addition, we

calculate a meta-feature vector describing the characteristics of each data set. The predictors learn to correlate specific characteristics with matching AL strategies. In order to recommend fitting AL strategies for an unseen data set, one must calculate its meta-feature vector and then the trained predictors will use that vector to suggest suiting AL strategy candidates. In this study, we have utilized two distinct predictors, a decision-tree-based and a vector-space-based model.

#### 3.1 Measurement of AL Performance

In order to determine when an AL strategy is advisable for a given input vector, the average integral over all time series for the specified AL strategy  $s$ , data set  $d$ , metric  $m$  and batch size  $b$  is calculated. We choose the integral as our scoring metric as it favours *monotonic increase*, *fast growth* and a *high long term value* which, when significant, results in a good performance of the metric with only a few labeled data points. Let  $T$  be the set of all time series sampled for the parameters  $s, d, m$  and

b. The average integral is calculated regarding:

$$\text{Int}(s,d,m,b) = \frac{\sum_{t \in T} \sum_{i \in \{0, \dots, 99\}} t_i}{|T|}$$

### 3.2 Performance Results

In **Figure 2**, the diagram shows the scores of the metric *performance* for its top 10 AL strategies on the *xor\_4x4* data set as an example. Apart from the specific ranking of the AL strategies, the distributions of the scores for other data sets and metrics are similar, in that the scores of the respective AL strategies are similar in magnitude.

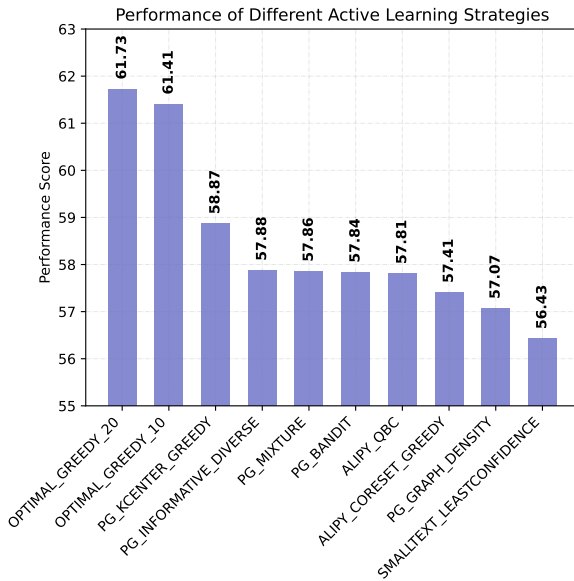


Figure 2: Calculated performance scores for the top 10 AL strategies of the *xor\_4x4* dataset, using the accuracy metric at batch size 1.

### 3.3 Characterizing Data Sets

In order to abstractly describe data sets on a structural level, we employ so called meta-features<sup>4</sup> which capture (1) *simple, statistical and information-theoretic characteristics* (e.g. mean value ( $\mu$ ), skewness, entropy) and (2) *concept and complexity characteristics* (e.g. feature correlation) (Brazdil et al., 2022). These meta-features are collected to a single meta-feature vector per data set. The predictive models learn patterns in the meta-feature vectors of the training data sets, establish correlations and make extrapolations to new, unlabeled data sets. We also used PCA to reduce the

<sup>4</sup>[https://github.com/paul1995tu/Data-Mining/blob/main/src/dataset\\_metafeatures/metrics.py](https://github.com/paul1995tu/Data-Mining/blob/main/src/dataset_metafeatures/metrics.py)

dimensions of the meta-feature vectors and measure how that influences the model’s performance. For in depth explanations, see **Section 3.4**.

### 3.4 Predictors

By using the integral and meta-feature vectors, it can be determined which AL strategies work well on specific data sets and how these data sets can be structurally described. The information is used to train the predictors, which learn to correlate data set characteristics with effective AL strategies, allowing recommendations to be made for unseen data sets. Therefore, decision-tree-based and vector-space-based models are proposed and evaluated. To simplify, the experiments were limited to 4 selected metrics: *accuracy*, *weighted F1-score*, *weighted precision*, and *weighted recall*. Additionally, the effects of applying PCA to the training meta-feature vectors were examined by evaluating the model’s performance across dimensions ranging from 1 to 9.

#### 3.4.1 Decision Trees

Decision trees are powerful and widely used ML algorithms that can be employed for both classification and regression tasks. In the context of this study, we used the scikit-learn implementation of classification trees, which is based on Quinlan and Hastie et al. (2009). A trained decision tree is structured as a flowchart-like model, where each internal node represents a meta-feature of the data sets, each branch a decision rule and each leaf node a recommended AL strategy.

#### 3.4.2 Vector Space

For the vector space model we consider the meta-feature vectors as part of an  $n$ -dimensional vector space, where similar meta-feature vectors are closer together while more distinct meta-feature vectors are not. In this work we use cosine-similarity as a measure of similarity for two vectors. Let  $V$  be the set of all training meta-feature vectors and  $S$  the set of all AL strategies. Regarding 3.1, there exists a score function  $\text{score}(s, v)$  which calculates the performance of a strategy  $s \in S$  on the underlying data set of a meta-feature vector  $v \in V$ . For an unseen data set meta-feature vector,  $\hat{v}$  the vector space model recommends the highest scoring AL strategy  $s_r \in S$  regarding the formula:

$$s_r = \arg \max_{s \in S} \left( \sum_{v \in V} \left( \frac{\hat{v} \cdot v}{\|\hat{v}\| * \|v\|} * \text{score}(s, v) \right) \right).$$

### 3.4.3 Evaluation of Predictors

For evaluation, the total 39 data sets are randomly split into a training class of 29 data sets and a test class of 10 data sets. For each data set of the test class, the predictor’s recommendation was compared to our pre-calculated rankings (c.f. 3.1). As **Figure 2** shows, the scores of the best AL strategies can possibly differ just marginally. Therefore, an AL strategy very close to the best one should still be considered as correctly recommended. To meet this requirement, we define that a prediction is considered to be correct if it is among the top 5 of the best performing AL strategies and performs not less than 2% as the best AL strategy.

### 3.4.4 Results

**Table 1** in the **Appendix** summarizes the calculated results of the vector spaces and the decision trees. The best performing predictor is the vector space with a dimension size reduced to 1. This system has an overall score of 68.24%. The best performing decision tree has an overall score of 55.62% for a reduction of the dimensions to 3. While the best performing vector space predictor has also the best performing metrics, the best performing decision tree has only the best performing F1-Score with 58.87%. For an in depth analysis of the scores for various dimensions, see **Table 1**.

### 3.4.5 Interpretation of Results

Since all evaluated predictors are trained on only about 75% of the given data set, it can be assumed that a fully trained predictor is likely to perform better. However, it can be assumed that the PCA-based dimension reduction yields biased results. This is because PCA is applied to the entire set of the evaluation data sets instead of the single evaluation data set in question. Therefore, the dimension reduction is based on additional information that normally would not be available.

## 4 Impact of First AL Cycle Value on Total Performance

To address the practical question of whether a high-performing time series at the beginning will continue to perform well overall, we explore the correlation between the score of the first AL cycle and overall performance. The goal is to estimate the probability of achieving a top-k result based on a given starting value in the first AL cycle. In this way, one can determine if the estimated probability

of obtaining a top-k result is not satisfactory and decide to randomly select alternative parameters for the time series to obtain a better chance of getting into the top-k than before.

### 4.1 Analysis of Performance Data

For analyzing the problem, the focus lies on the 4 performance metrics *accuracy*, *weighted F1-Score*, *weighted recall* and *weighted precision*. In order to obtain insights into the start performance, the distribution of the scores reached at the first AL cycle is considered individually for each batch-size.

### 4.2 Results

A sample distribution is illustrated in **Figure 3** for batch-size 1 and metric *accuracy*. It can be seen that about 32.6% of values are in the range of [0, 0.5], while the majority (67.4%) are in the range of (0.5, 1.0]. Similar distributions are given for the other 3 metrics and their batch-sizes with the exception of the metric *weighted precision* whose distribution is more leaned to the right as shown in **Figure 4**. For this case only 22.7% of values are in the range of [0, 0.5] and the remaining 77.3% in the range of (0.5, 1.0]. Additionally, we closely study the top 5% ( $\approx 20,000$ ) of best-performing time series across each metric and batch-size. The investigation of the question to which first cycle value a final value belongs, leads to **Figure 5** for the metric *weighted precision* and batch-size 1. The bar chart shows that about 6.2% of all time series of the top 20,000 have a starting value in the range of [0, 0.8], whereas the remaining 93.8% have a starting value of 0.8 or above.

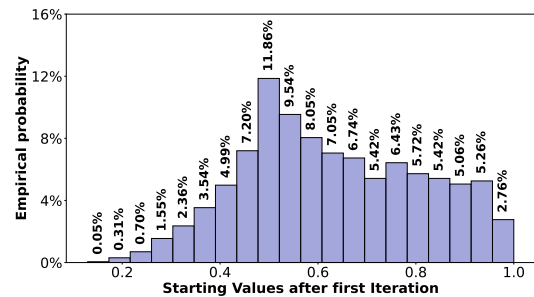


Figure 3: Distribution of Starting Values for metric accuracy and batch-size 1.

### 4.3 Interpretation of Results

The observations indicate that a high starting value tends to result in a high end value. In particular, the fact that the lowest starting point of the best 20,000

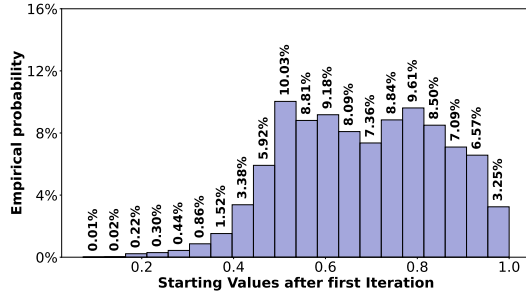


Figure 4: Distribution of Starting Values for weighted precision and batch-size 1.

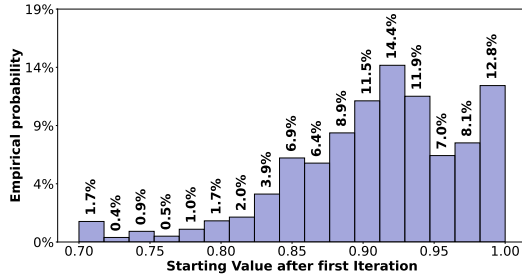


Figure 5: Distribution of Top 20,000 Starting Values for weighted precision and batch-size 1.

results is just above 0.7 supports these conclusions. It should also be noted that a series with a low initial value can result in a high final value. However there is no clear evidence of frequent occurrence of this time series behavior based on our results.

## 5 Conclusion

In our work, we explore various AL strategies, analyzing their characteristics and comparing them through clustering. Notably, we identify similarities and common patterns among these AL strategies. Furthermore, we have developed and evaluated a recommendation framework consisting of two predictive models: a decision-tree-based model and a vector-space-based model. These models are designed to recommend active learning strategies based on data set characteristics and input vectors. Additionally, we have examined the influence of the first AL-cycle performance on the overall performance for gaining insights into the behaviour of time series of AL strategies.

## 6 Future Work

It is crucial to recognize and address the limitations inherent in our study, including the relatively small sample size of the data sets employed to evaluate

the recommendation framework. It would be interesting to analyse the performance of the recommendation models trained on more data sets since we expect better results. It is also worth noting that we have employed a single suitable score to measure the performance of an AL strategy. This does not exclude the possibility that there are other scores that lead to slightly different results. In particular, a score that more clearly interprets the subtle differences between individual time series could help to improve the results. Further it would be interesting to develop a probabilistic framework which is capable of calculating the probability that an AL-strategy with a first cycle value of  $x$  tends to achieve a final score in  $[a, b]$ , given a specific learner model, batch-size and metric.

## References

- Pavel Brazdil, Jan N van Rijn, Carlos Soares, and Joaquin Vanschoren. 2022. *Metalearning: applications to automated machine learning and data mining*. Springer.
- Julius Gonsior, Maik Thiele, and Wolfgang Lehner. 2022. Imital: Learned active learning strategy on synthetic data. In *International Conference on Discovery Science*, pages 47–56. Springer.
- Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. 2009. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer.
- Daniel Kottke, Marek Herde, Tuan Pham Minh, Alexander Benz, Pascal Mergard, Atal Roghman, Christoph Sandrock, and Bernhard Sick. 2021. scikit-activeml: A library and toolbox for active learning algorithms.
- J Ross Quinlan. Program for machine learning. *Mach Learn*, 16.
- Christopher Schröder, Lydia Müller, Andreas Niekler, and Martin Potthast. 2023. Small-text: Active learning for text classification in python. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 84–95. Association for Computational Linguistics.
- Ying-Peng Tang, Guo-Xiang Li, and Sheng-Jun Huang. 2019. Alipy: Active learning in python. *arXiv preprint arXiv:1901.03802*.
- Yao-Yuan Yang, Shao-Chuan Lee, Yu-An Chung, Tung-En Wu, Si-An Chen, and Hsuan-Tien Lin. 2017. libact: Pool-based active learning in python. *arXiv preprint arXiv:1710.00379*.

## A Appendix

| Vector Space<br>Runs       | Overall<br>Scores | Metrics             | Score        | Decision Tree<br>Runs       | Overall<br>Scores | Metrics             | Score        |
|----------------------------|-------------------|---------------------|--------------|-----------------------------|-------------------|---------------------|--------------|
| Without PCA                | 64.66             | Accuracy            | 65.21        | Without PCA                 | 51.44             | Accuracy            | 52.94        |
|                            |                   | w. F1-Score         | 69.04        |                             |                   | w. F1-Score         | 51.93        |
|                            |                   | w. Precision        | 58.84        |                             |                   | w. Precision        | 41.49        |
|                            |                   | w. Recall           | 65.54        |                             |                   | w. Recall           | 59.41        |
| <b>PCA 1<br/>Dimension</b> | <b>68.24</b>      | <b>Accuracy</b>     | <b>69.19</b> | PCA 1<br>Dimension          | 54.54             | Accuracy            | 55.53        |
|                            |                   | <b>w. F1-Score</b>  | <b>72.58</b> |                             |                   | w. F1-Score         | 57.07        |
|                            |                   | <b>w. Precision</b> | <b>62.46</b> |                             |                   | <b>w. Precision</b> | <b>49.71</b> |
|                            |                   | <b>w. Recall</b>    | <b>68.74</b> |                             |                   | w. Recall           | 56.57        |
| PCA 2<br>Dimensions        | 67.78             | Accuracy            | 68.34        | PCA 2<br>Dimensions         | 54.46             | Accuracy            | 55.00        |
|                            |                   | w. F1-Score         | 72.26        |                             |                   | w. F1-Score         | 58.11        |
|                            |                   | w. Precision        | 62.26        |                             |                   | w. Precision        | 47.83        |
|                            |                   | w. Recall           | 68.24        |                             |                   | w. Recall           | 59.23        |
| PCA 3<br>Dimensions        | 66.86             | Accuracy            | 67.25        | <b>PCA 3<br/>Dimensions</b> | <b>55.62</b>      | Accuracy            | 56.30        |
|                            |                   | w. F1-Score         | 71.11        |                             |                   | <b>F1-Score</b>     | <b>58.87</b> |
|                            |                   | w. Precision        | 61.01        |                             |                   | w. Precision        | 48.62        |
|                            |                   | w. Recall           | 68.10        |                             |                   | w. Recall           | 61.63        |
| PCA 4<br>Dimensions        | 65.70             | Accuracy            | 66.22        | PCA 4<br>Dimensions         | 54.78             | <b>Accuracy</b>     | <b>56.76</b> |
|                            |                   | w. F1-Score         | 69.72        |                             |                   | w. F1-Score         | 58.16        |
|                            |                   | w. Precision        | 59.82        |                             |                   | w. Precision        | 45.74        |
|                            |                   | w. Recall           | 67.12        |                             |                   | w. Recall           | 63.50        |
| PCA 5<br>Dimensions        | 65.45             | Accuracy            | 66.02        | PCA 5<br>Dimensions         | 53.82             | Accuracy            | 56.02        |
|                            |                   | w. F1-Score         | 70.01        |                             |                   | w. F1-Score         | 57.24        |
|                            |                   | w. Precision        | 59.77        |                             |                   | w. Precision        | 45.57        |
|                            |                   | w. Recall           | 66.03        |                             |                   | <b>w. Recall</b>    | <b>64.86</b> |
| PCA 6<br>Dimensions        | 65.61             | Accuracy            | 66.71        | PCA 6<br>Dimensions         | 53.30             | Accuracy            | 54.61        |
|                            |                   | w. F1-Score         | 69.69        |                             |                   | w. F1-Score         | 57.39        |
|                            |                   | w. Precision        | 59.69        |                             |                   | w. Precision        | 44.55        |
|                            |                   | w. Recall           | 66.36        |                             |                   | w. Recall           | 64.36        |

Table 1: Performance of the evaluated predictors. All scores show the accuracy in percentage. Each type of recommendation system is build multiple times based on different dimensional reductions. The overall best performing predictor, which has the highest overall scores, as well as scores in the metrics accuracy, weighted

F1-Score, weighted precision and weighted recall, is a vector space predictor build on a one-dimensional meta-feature vector. The overall best performing decision tree predictor is build on a 3 dimensional meta-feature vector. The decision tree predictors build on 1, 3, 4 and 5 dimensions are performing best on individual different metrics.